

Introducción a VHDL. Ejercicio Final

Implemente en VHDL un filtro digital dado por la siguiente ecuación:

$$y[n] = 0,0675 x[n] + 0,1349 x[n - 1] + 0,0675 x[n - 2] + 1,1430 y[n - 1] - 0,4128 y[n - 2]$$

Que describe un filtro IIR pasa-bajos con frecuencia de corte normalizada 0,2 (Por ejemplo, para una frecuencia de muestreo de 1000 Hz resulta una frecuencia de corte de 100 Hz).

Para la implementación primero convierta los coeficientes en números enteros multiplicando cada uno por 2^{11} , redondeando al entero más cercano y codificando el número entero resultante en CA2. Considerando que los datos de entrada vienen cuantizados en CA2 de 12 bits calculé la cantidad de bits necesarios en cada operación, suponiendo que se desea una salida de 32 bits. Para gestionar los datos de entrada incluya una señal "data_valid" que esté en alto en cada ciclo de reloj que hay una nueva muestra de entrada disponible.

Desarrolle la implementación sin utilizar las librerías aritméticas de VHDL, implementando un pipeline que solo avance si la señal de data_valid esté activa. Para las sumas puede implementar arquitectura Ripple Carry, o la que le resulte más adecuada. Para las multiplicaciones puede utilizar algoritmo de Booth o codificarlas con bitshifts y sumas, aprovechando que los coeficientes son constantes y conocidos.

Escriba un Testbench para el módulo y pruébelo con dos señales sinusoidales: Una en la banda de paso del filtro y una en la banda de rechazo. Como referencia puede usar que una onda sinusoidal de 10 muestras por ciclo está justo la frecuencia de corte del filtro. Para visualizar las señales en modelsim conviene usar format -> analógic. Para generar las señales sinusoidales puede usar una sintaxis como la siguiente:

```
-- Definimos un arreglo con los estímulos
type stim_array is array (natural range <>) of std_logic_vector(11 downto 0);

constant stim_4 : stim_array := (
    std_logic_vector(to_signed( 0, 12)),
    std_logic_vector(to_signed(2047, 12)),
    std_logic_vector(to_signed(0, 12)),
    std_logic_vector(to_signed(-2047, 12))|
);
```

Determine la máxima frecuencia de operación del circuito y el uso de recursos.

BONUS: Realice una segunda implementación controlada por una máquina de estados finitos (FSM), de modo que los cálculos se dividan en varias etapas secuenciales. Verifique cómo este enfoque permite aumentar la frecuencia máxima de reloj al reducir el camino crítico, a costa de que no sea posible procesar una muestra en cada ciclo de reloj. Compare ambas implementaciones en términos de frecuencia máxima alcanzable, área de hardware y tasa de datos a la salida.